

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Azalea

***shell gráfico multicompositor para
Wayland usando o protocolo layer shell***

Enzo Hideo Kobayashi

MONOGRAFIA FINAL

MAC 499 — TRABALHO DE
FORMATURA SUPERVISIONADO

Supervisor: Prof. Alfredo Goldman
Cossupervisor: Prof. Nelson Posse Lago

São Paulo
2025

*O conteúdo deste trabalho é publicado sob a licença CC BY 4.0
(Creative Commons Attribution 4.0 International License)*

Resumo

Enzo Hideo Kobayashi. **Azalea: *shell* gráfico multicompositor para Wayland usando o protocolo *layer shell***. Monografia (Bacharelado). Instituto de Matemática e Estatística, Universidade de São Paulo, São Paulo, 2025.

Shell gráfico é um programa responsável pelas interfaces gráficas que permitem um usuário interagir com o sistema operacional. Isso inclui as barras de tarefas, menu de início, papel de parede, barra de busca, menus de configuração, etc. Normalmente, os *shells* gráficos estão fortemente acoplados a um ambiente de área de trabalho, como o plasmashell do KDE e o GNOME Shell, tornando quase inviável trocar o compositor ou reusar o *shell* gráfico em outro ambiente. Nesse sentido, o objetivo deste projeto foi desenvolver um *shell* gráfico que funcione em múltiplos compositores Wayland diferentes por meio do protocolo *layer shell*.

O software foi desenvolvido em Rust usando Relm4 (GTK4) como biblioteca gráfica e Nix para criar ambientes de desenvolvimento reprodutíveis, empacotar a aplicação e testar a sua execução em diferentes compositores Wayland por meio de máquinas virtuais NixOS. O projeto também pode ser utilizado como biblioteca, permitindo que o usuário crie seus próprios componentes gráficos.

Palavras-chave: Linux. Wayland. protocolo *layer shell*. *shell* gráfico. Rust. GTK4. Relm4. Nix.

Abstract

Enzo Hideo Kobayashi. ***Azalea: compositor-agnostic graphical shell for Wayland using the layer shell protocol.*** Capstone Project Report (Bachelor). Institute of Mathematics and Statistics, University of São Paulo, São Paulo, 2025.

Graphical shell is a program responsible for the graphical interfaces that allow a user to interact with the operating system. This includes taskbars, start menu, wallpaper, search bar, settings menus, etc. Usually, graphical shells are strongly coupled with a desktop environment, such as plasmashell from KDE and GNOME Shell, rendering it almost infeasible to replace the compositor or reuse the graphical shell in another environment. In view of that, the objective of this project is to develop a graphical shell that works in multiple different Wayland compositors through the layer shell protocol.

The software was developed in Rust using Relm4 (GTK4) as a graphical library and Nix to create reproducible development environments, package the application, and test its execution in different Wayland compositors using NixOS virtual machines. The project can be used as a library as well, allowing the user to create their own graphical components.

Keywords: Linux. Wayland. layer shell protocol. graphical shell. Rust. GTK4. Relm4. Nix.

Lista de abreviaturas

D-Bus	Desktop Bus
MPRIS	Media Player Remote Interfacing Specification
MVU	Model-View-Update
LSP	Language Server Protocol
RAII	Resource acquisition is initialization
UUID	Universally Unique Identifier
SASS	Syntactically Awesome Style Sheets
CSS	Cascading Style Sheets
JSON	JavaScript Object Notation
RON	Rusty Object Notation
ALSA	Advanced Linux Sound Architecture
ACPI	Advanced Configuration and Power Interface
DDC/CI	Display Data Channel/Command Interface

Lista de figuras

- 2.1 Lista de compositores com suporte ao protocolo *layer shell* (WAYLAND EXPLORER, 2025) 4
- 2.2 Representação visual das camadas e âncoras do protocolo *layer shell* 4
- 4.1 Papel de parede padrão¹ 18
- 4.2 *Slider* de volume 20
- 4.3 *Slider* de brilho 20
- 4.4 *Widget* de data e hora 21
- 4.5 *Widget* do menu de início. 22
- 4.6 *Widget* de controle de dispositivos *bluetooth* 22
- 4.7 *Widget* de controle de conexões rede 23
- 4.8 *Widget* de notificações 24
- 4.9 *Widget* MPRIS 25
- 4.10 *Widgets* de *shortcut* para Steam, Chromium e Firefox 25
- 4.11 *Widget* de busca de aplicações 26
- 4.12 Colagem² de fotos com todos os *widgets* 27
- 5.1 Azalea rodando no Sway em máquina virtual NixOS. 31
- 5.2 Azalea rodando no Hyprland em máquina virtual NixOS. 31
- 5.3 Azalea rodando no Wayfire em máquina virtual NixOS. 32
- 5.4 Azalea rodando no Miracle em máquina virtual NixOS. 32
- 5.5 Azalea rodando no Niri em máquina virtual NixOS. 35

¹ ilustração feita pelo autor

² Tecnicamente, não é possível mostrar todos os *widgets* ao mesmo tempo, pois o GTK fecha os *popups* quando eles perdem foco. Nesse sentido, a imagem é uma colagem de fotos dos *widgets*.

Lista de programas

3.1	Exemplo de macros procedurais em Rust.	8
3.2	Exemplo de macro simples em Rust.	9
3.3	Exemplo básico de uso da biblioteca gtk4-layer-shell	10
3.4	Exemplo de uso da biblioteca Relm4 para fazer um <i>widget</i> “contador” de apenas um botão, adaptado de <i>GUI development with Relm4</i> , Cap. 2 (RELM4 CONTRIBUTORS, 2025).	11
3.5	Exemplo de <i>proxy</i> usando zbus.	12
3.6	Exemplo de daemon de notificação implementado com zbus.	12
3.7	Exemplo de estilo Sass usado no Azalea.	13
3.8	Exemplo de uso da macro <i>derive StaticServiceManager</i> para implementar um gerenciador de serviço <i>singleton</i>	15
4.1	Exemplo de configuração do papel de parede em formato RON.	18
4.2	Exemplo de configuração da barra de tarefas do canto inferior da tela em formato RON.	19
4.3	Exemplo de configuração do <i>widget</i> de data e hora RON.	21
4.4	Exemplo de configuração de <i>shortcut</i> em formato RON.	26
5.1	Exemplo de como usar um <i>dev shell</i> nix.	29
5.2	Exemplo de <i>dev shell</i> nix.	30
5.3	Exemplo simples de como empacotar uma aplicação fantasia em nix. . .	33
5.4	Exemplo de teste em máquina virtual NixOS	34

Sumário

1	Introdução	1
2	Conceitos fundamentais	3
2.1	Shell	3
2.2	Wayland	3
2.2.1	Protocolo <i>layer shell</i>	4
2.3	D-Bus	5
3	Detalhes de implementação	7
3.1	Ferramentas	7
3.1.1	Rust	7
3.1.2	Relm4, GTK4 e gtk4-layer-shell	8
3.1.3	Zbus	10
3.1.4	Sass	10
3.2	Arquitetura	10
3.2.1	Cliente-Servidor	10
3.2.2	Serviços	13
4	Funcionalidades de <i>desktop</i>	17
4.1	Papel de parede	18
4.2	Barra de tarefas	19
4.2.1	Controle de volume	20
4.2.2	Controle de brilho	20
4.2.3	Data e hora	20
4.2.4	Menu de início	21
4.2.5	Controle de dispositivos <i>bluetooth</i>	21
4.2.6	Controle de conexões de rede	22
4.2.7	Notificações	24
4.2.8	Controle de mídia	24

4.2.9	<i>Shortcuts</i>	25
4.2.10	Barra de busca de aplicações	25
4.2.11	<i>System tray</i>	26
4.3	<i>Overview</i>	27
5	Nix	29
5.0.1	Ambiente de desenvolvimento	29
5.0.2	Empacotamento	29
5.0.3	Testes em máquinas virtuais NixOS	30
6	Conclusão	37
	Referências	39

Capítulo 1

Introdução

Um dos pontos fortes do sistema operacional Linux é a sua modularidade, no sentido de permitir ao usuário escolher e trocar vários componentes do sistema. Desde gerenciadores de login, ambientes de área de trabalho, gerenciadores de janela, shells de linha de comando, até o sistema de *init*.

No entanto, o mesmo não pode ser necessariamente dito dos shells gráficos. Há um forte acoplamento entre o shell gráfico e o gerenciador de janela, ou compositor. Por exemplo, não é possível utilizar o GNOME Shell em outro ambiente que não seja o do Mutter, o compositor Wayland do GNOME.

Nesse sentido, o objetivo deste trabalho é desenvolver um shell gráfico que funcione em vários compositores Wayland diferentes. E o programa deve implementar funcionalidades essenciais de um *desktop*, como:

- Papel de parede
- Barra de tarefas customizável
- Controle de volume
- Controle de brilho
- Controle de dispositivos bluetooth
- Controle de conexões de rede
- Data e hora
- Notificações
- Controle de mídia
- Menu de início
- *Shortcuts* para abrir aplicações
- Barra de busca de aplicações
- *System tray*

O próximo capítulo explica conceitos fundamentais para compreender o projeto. O terceiro capítulo trata das ferramentas utilizadas e detalhes gerais de implementação. O quarto capítulo fala sobre as funcionalidades implementadas. O quinto capítulo trata do uso do Nix no projeto. Por fim, o último capítulo conclui com os resultados alcançados e outros protocolos Wayland de interesse que não foram tratados neste trabalho.

Capítulo 2

Conceitos fundamentais

O programa se trata de um *shell* gráfico para Wayland que utiliza o protocolo *layer shell* para funcionar em múltiplos compositores Wayland e usa D-Bus para a implementação de diversas funcionalidades de um *desktop*.

Este capítulo é dedicado a explicar cada um desses termos e discutir as ferramentas utilizadas.

2.1 Shell

O *shell* é um programa que permite ao usuário interagir com o sistema, atuando como uma “camada” entre o usuário e o sistema operacional (WIKIPEDIA CONTRIBUTORS, 2025).

Existem *shells* de linha de comando como *bash* e *zsh*, mas também existem os *shells* gráficos como o *plasmashell*, *GNOME Shell* e este projeto, o *Azalea*. O *shell* gráfico é responsável, basicamente, por todas as interfaces gráficas usadas para interagir com o sistema.

2.2 Wayland

Wayland é um protocolo de display criado para substituir o X11 (FREEDESKTOP, 2025b). No X11, o compositor e gerenciador de janelas são programas distintos, porém no caso do Wayland o compositor é responsável tanto pelo gerenciamento de janelas quanto da composição. Isto é, o compositor é responsável por decidir o posicionamento das janelas e pode aplicar efeitos, como *shaders* e animações.

O compositor Wayland atua como servidor de display e cada aplicação que deseja criar uma janela atua como cliente. Os protocolos Wayland especificam como essa comunicação entre cliente e servidor deve acontecer. Existem os protocolos *core*, que todo compositor Wayland deve implementar, e existem os protocolos de extensão, que são opcionais e estendem a funcionalidade do compositor. Um desses protocolos de extensão é o protocolo *layer shell* ou, mais precisamente, *wlr-layer-shell-unstable-v1*.

2.2.1 Protocolo *layer shell*

Protocolos Wayland passam por três fases principais: experimental ou *unstable*, *staging* e *stable*. A fase experimental é a fase em que o protocolo ainda está em desenvolvimento. Quando o protocolo alcança certa maturidade e reconhecimento, ele passa para a fase de teste ou *staging* em que ele é incluído no repositório principal (<https://gitlab.freedesktop.org/wayland/wayland-protocols>) e os compositores e clientes Wayland são incentivados a implementar o protocolo para fins de teste. Por fim, após ser suficientemente testado pela comunidade, ele pode ser considerado estável (FREEDESKTOP, 2025c).

O protocolo *layer shell* é experimental e surgiu inicialmente no wlroots, uma biblioteca de criação de compositores Wayland. Já que essa biblioteca é utilizada por vários compositores, o protocolo *layer shell* é amplamente suportado apesar de ser experimental (2.1).

Compositor Support															
Mutter	Muffin	KWin	Sway	COSMIC	Hyprland	niri	Weston	Labwc	Cage	Wayfire	GameScope	Jay	Mir	Treeland	Louvre
48.0	...	6.4	1.11	...	0.48	25.08	14	0.9.0	0.2.0	0.9.0	3.15.14	1.11.0	2.19	0.6.1	2.14.1
zwlr_layer_shell_v1	x	x	5	4	4	5	4	x	4	x	4	4	5	4	5

Figura 2.1: Lista de compositores com suporte ao protocolo *layer shell* (WAYLAND EXPLORER, 2025)

Esse protocolo permite que uma aplicação cliente peça ao compositor em qual camada a sua janela deve ser renderizada e em qual canto da tela deve ser ancorada (2.2).

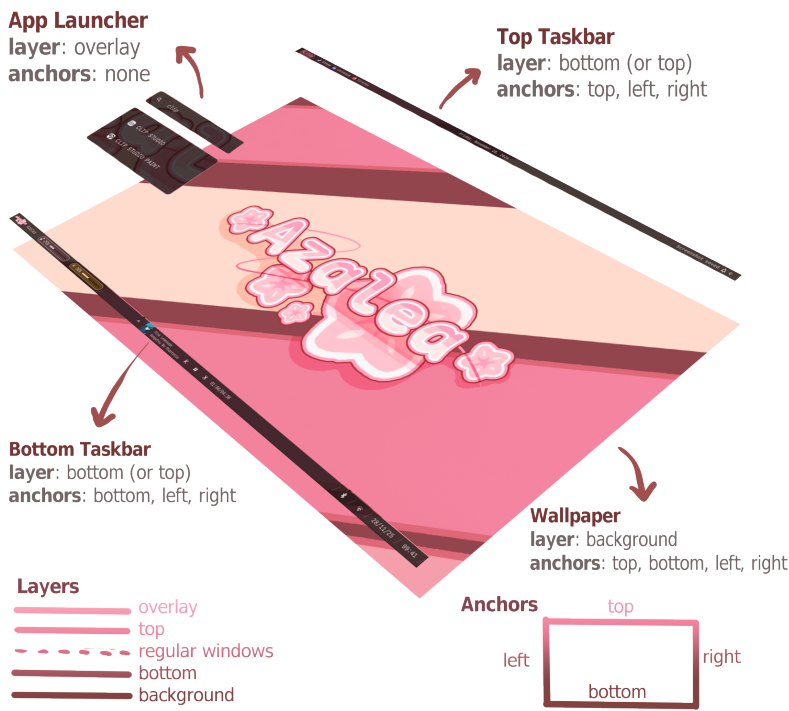


Figura 2.2: Representação visual das camadas e âncoras do protocolo *layer shell*

O cliente também pode pedir ao compositor uma “zona exclusiva” para evitar que sua janela intersecte outras janelas. Isso é particularmente útil em barras de tarefas para evitar que ela sejam sobrepostas por outras janelas. No entanto, esse comportamento depende da implementação, então o resultado pode ser um pouco diferente dependendo do compositor.

2.3 D-Bus

D-Bus, ou *Desktop Bus*, é um sistema de barramento de mensagens que permite comunicação entre processos. Seu principal uso, como o nome sugere, é conectar os diversos componentes e aplicações utilizados em um *desktop*.

Normalmente, são utilizados dois daemons de barramento: sistema e sessão. O barramento de sistema é utilizado para comunicação que abrange o sistema como um todo, como o *Network Manager*. Enquanto que o barramento de sessão é utilizado apenas na sessão do usuário, como o *MPRIS*.

Aplicações clientes D-Bus se conectam a um desses barramentos criando uma conexão de nome único. Essa conexão pode expor vários “objetos” e cada objeto pode implementar várias interfaces, que definem os métodos, propriedades e sinais que o objeto expõe. Os métodos permitem receber e retornar valores numéricos, strings, arrays, dicionários e outras estruturas mais complexas. As propriedades são valores do objeto que podem permitir leitura e escrita, apenas leitura, ou apenas escrita. Os sinais de um objeto permitem uma comunicação de tipo *publish-subscribe*, em que os objetos podem fazer *broadcast* de um sinal e clientes interessados podem se inscrever para serem alertados quando esse sinal é emitido (FREEDESKTOP, 2025a).

Em termos de implementação, há dois tipos principais de uso do D-Bus: os serviços e os clientes, ou *proxies*. Uma aplicação serviço é aquela que se conecta ao barramento com um nome específico e fornece objetos que implementam alguma interface. Aplicações clientes que desejam usar esse serviço precisam implementar um *proxy*, uma estrutura de dados local no programa que representa esse objeto e implementa suas interfaces, de modo que uma chamada de método nesse *proxy* corresponderia a chamar o método do objeto D-Bus.

Capítulo 3

Detalhes de implementação

3.1 Ferramentas

O programa foi escrito em Rust utilizando Relm4 (GTK4) como biblioteca gráfica, gtk4-layer-shell para usar o protocolo *layer shell* e zbus para implementar os serviços e *proxies* D-Bus. Outras bibliotecas de usos específicos são mencionadas nas seções seguintes quando tal uso for explicado.

3.1.1 Rust

Rust (<https://rust-lang.org/>) é uma linguagem de programação de tipagem forte e estática. Um de seus pontos principais é a segurança de memória, alcançada por meio de seu sistema de *ownership*: todo valor possui um único dono e quando esse dono sai do escopo, o valor é destruído.¹ Nesse sentido, Rust não possui um coletor de lixo, a memória é gerenciada em tempo de compilação de forma similar à técnica de RAIL.

Uma funcionalidade Rust utilizada amplamente no projeto são as macros procedurais de tipo *derive* e *attribute*, que permitem a implementação automática de *traits*² e atributos customizados, respectivamente. O exemplo em 3.1 demonstra o uso da macro *derive* para implementar o *trait Default* e uma macro atributo para anotar qual membro da enumeração deve ser usado como valor padrão.

Também existem as macros não-procedurais, que atuam como uma função, mas permitem argumentos de tipos diferentes e quantidades variáveis de argumentos. O exemplo 3.2 demonstra um uso básico em que a macro `my_macro` aceita ser chamada de três formas diferentes:

- `my_macro!(arg: literal):` Imprime o literal
- `my_macro!(arg1: literal, arg2: literal):` Imprime os dois literais

¹ Uma variável pode ser destruída antes que ela saia de escopo por meio da função *drop*. *The Rust Programming Language, Third Edition*, Cap. 4 (STEVE KLABNIK, CAROL NICHOLS, CHRIS KRYCHO, RUST COMMUNITY CONTRIBUTORS, 2025)

² *Traits* em Rust são como interfaces, eles definem quais métodos devem ser implementados

- `my_macro!(arg: type)`: Implementa o *trait Display* para um tipo qualquer

Programa 3.1 Exemplo de macros procedurais em Rust.

```

1  // Macro derive que implementa os traits Default, Debug e PartialEq
2  #[derive(Default, Debug, PartialEq)]
3  enum Animal {
4      // Macro atributo que anota este valor como padrão
5      //
6      // Isso é usado pela macro derive para implementar o trait Default
7      #[default]
8      Cat,
9
10     Dog,
11 }
12
13 // A implementação manual do trait Default seria:
14 // impl Default for Animal {
15 //     fn default() -> Self {
16 //         Animal::Cat
17 //     }
18 // }
19
20 fn main() {
21     // Já que o trait Default foi implementado, podemos usar a função
22     // default() para criar um Animal padrão.
23     let animal = Animal::default();
24
25     // Verifica que o animal criado é de fato o valor padrão Animal::Cat
26     assert_eq!(animal, Animal::Cat); // Necessita os traits Debug e PartialEq
27 }

```

3.1.2 Relm4, GTK4 e gtk4-layer-shell

GTK4 (<https://www.gtk.org/>) é um *toolkit* de *widgets* de código aberto do GNOME que permite a criação de aplicações gráficas. A biblioteca é *single-threaded*, no sentido que o laço principal que cuida dos eventos da aplicação e os *widgets* devem estar na mesma *thread*, não é possível enviar objetos GTK entre *threads*.

Relm4 (<https://relm4.org/>) é uma biblioteca que simplifica o uso do GTK4 em Rust com base na arquitetura Elm (*Model-View-Update*). Essa arquitetura consiste em três componentes principais:

- *Model*: conjunto de dados que define o estado do *widget*
- *View*: representação gráfica do modelo
- *Update*: define como o modelo deve ser atualizado

O exemplo 3.4 demonstra como essa arquitetura é utilizada para criar *widgets*.

A biblioteca `gtk4-layer-shell` adiciona suporte ao protocolo *layer shell* ao GTK4. O exemplo 3.3 mostra como configurar as camadas, âncoras e zona exclusiva nas janelas criadas pelo GTK.

Programa 3.2 Exemplo de macro simples em Rust.

```

1  // Declaração da macro
2  #[macro_export]
3  macro_rules! my_macro {
4      ($arg:literal) => {
5          println!("Literal: {}", $arg);
6      };
7      ($arg1:literal, $arg2:literal) => {
8          println!("Two literals: {} {}", $arg1, $arg2);
9      };
10     ($arg:ty) => {
11         impl std::fmt::Display for $arg {
12             fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
13                 write!(f, "Generic display: {}", std::any::type_name::<Self>())
14             }
15         }
16     };
17 }
18
19 // Enumeração com valores internos
20 enum Color {
21     RGB(u8, u8, u8),
22     BGR(u8, u8, u8),
23     RGBA(u8, u8, u8, u8),
24     ComplexRgb {
25         r: u8,
26         g: u8,
27         b: u8,
28     }
29 }
30
31 // Essa chamada implementa o trait de Display para a struct Color
32 my_macro!(Color);
33
34 fn main() {
35     my_macro!("hello world");
36     my_macro!(3, 3);
37
38     // O trait de Display permite que instâncias da struct sejam transformadas
39     // em string e imprimidas na tela pela macro println
40     println!("{}", Color::RGB(0, 0, 255));
41
42     // output:
43     //
44     // Literal: hello world
45     // Two literals: 3 3
46     // Generic display: main::Color
47 }

```

Programa 3.3 Exemplo básico de uso da biblioteca gtk4-layer-shell

```

1  let window = gtk::Window::new();
2
3  window.init_layer_shell();
4  window.set_namespace(Some("taskbar"));
5  window.set_keyboard_mode(gtk4_layer_shell::KeyboardMode::OnDemand);
6  window.set_exclusive_zone(-1);
7  window.set_layer(gtk4_layer_shell::Layer::Overlay);
8  window.set_anchor(gtk4_layer_shell::Edge::Top, true);

```

3.1.3 Zbus

Zbus (<https://z-galaxy.github.io/zbus/>) é uma biblioteca em Rust que permite a criação de serviços e proxies D-Bus por meio de macros procedurais. O programa 3.5 mostra um exemplo de *proxy* para a interface D-Bus MPRIS e 3.6 mostra a implementação de um *daemon* de notificações. As macros procedurais tornam a implementação consideravelmente mais simples.

3.1.4 Sass

Sass é uma linguagem que estende o CSS e foi utilizada no projeto para estilizar visualmente o programa. Já que a biblioteca GTK4 já possui suporte para CSS, bastou compilar o arquivo de Sass para CSS. Para isso, foi utilizada a biblioteca *grass* (<https://crates.io/crates/grass>).

O usuário pode carregar um estilo customizado por meio do arquivo de configuração em “HOME/.config/azalea/style.scss” (3.7).

3.2 Arquitetura

3.2.1 Cliente-Servidor

O programa possui dois modos de execução: servidor e cliente, dependendo do comando:

- `azalea daemon start`: Inicia o servidor
- `azalea daemon stop`: Para o servidor
- `azalea window uuid`: Lista os UUIDs de todos os *widgets*
- `azalea window toggle [UUID]`: Mostrar ou esconder um *widget*
- `azalea window create [NOME]`: Cria um *widget* a partir de seu nome
- `azalea layer toggle [NAMESPACE]`: Mostrar ou esconder todos os *widgets* em camadas pertencentes a um certo *namespace*
- `style reload`: Recarregar estilo Sass

Programa 3.4 Exemplo de uso da biblioteca Relm4 para fazer um *widget* “contador” de apenas um botão, adaptado de *GUI development with Relm4*, Cap. 2 (REL4 CONTRIBUTORS, 2025).

```

1  struct Model {
2      counter: u8,
3  }
4
5  enum Input {
6      Increment,
7  }
8
9  #[relm4::component]
10 impl relm4::SimpleComponent for Model {
11     type Init = u8; // dados de inicialização do modelo
12     type Input = Input;
13     type Output = ();
14
15     view! {
16         gtk::Window {
17             gtk::Button {
18                 // Quando o botão é clicado, a mensagem de Input::Increment é
19                 // gerada
20                 connect_clicked => Input::Increment
21
22                 // Quando o Model atualizar, este valor também será atualizado
23                 #[watch]
24                 set_label: &format!("Counter: {}", model.counter),
25             },
26         }
27
28     fn update(&mut self, message: Self::Input, _sender: ComponentSender<Self>)
29     {
30         match message {
31             Input::Increment => {
32                 self.counter = self.counter.wrapping_add(1);
33             }
34         }
35
36         // Construtor
37         fn init(
38             init: Self::Init,
39             root: Self::Root,
40             sender: ComponentSender<Self>,
41         ) -> ComponentParts<Self> {
42             let model = Model { counter: init };
43             let widgets = view_output!();
44             ComponentParts { model, widgets }
45         }
46     }

```

Programa 3.5 Exemplo de *proxy* usando zbus.

```

1  #[derive(Clone, Debug, Default, serde::Serialize, serde::Deserialize,
    OwnedValue)]
2  #[zvariant(signature = "s")]
3  pub enum PlaybackStatus {
4      Playing,
5      Paused,
6      #[default]
7      Stopped,
8  }
9
10 #[proxy(
11     interface = "org.mpris.MediaPlayer2.Player",
12     default_path = "/org/mpris/MediaPlayer2"
13 )]
14 pub trait Player {
15     // Método do objeto D-Bus
16     fn play_pause(&self) -> zbus::Result<>;
17
18     // Propriedade do objeto D-Bus
19     #[zbus(property)]
20     fn playback_status(&self) -> zbus::fdo::Result<PlaybackStatus>;
21
22     // Sinal emitido pelo objeto D-Bus
23     #[zbus(signal)]
24     fn seeked(&self, position: i64) -> zbus::Result<>;
25 }

```

Programa 3.6 Exemplo de daemon de notificação implementado com zbus.

```

1  #[derive(Debug, zbus::zvariant::Type, serde::Serialize, serde::Deserialize)]
2  pub struct ServerInformation {
3      pub name: String,
4      pub vendor: String,
5      pub version: String,
6      pub spec_version: String,
7  }
8
9  pub struct Notifications { }
10
11 /// https://specifications.freedesktop.org/notification/1.3/protocol.html
12 #[zbus::interface(name = "org.freedesktop.Notifications")]
13 impl Notifications {
14     fn get_server_information(&self) -> zbus::fdo::Result<ServerInformation> {
15         Ok(ServerInformation {
16             name: format!(env!("CARGO_PKG_NAME")),
17             vendor: format!(env!("CARGO_PKG_AUTHORS")),
18             version: format!(env!("CARGO_PKG_VERSION")),
19             spec_version: format!("1.3"),
20         })
21     }
22 }

```

Programa 3.7 Exemplo de estilo Sass usado no Azalea.

```

1  :root {
2    --background: #191112;
3    --error: #ffb4ab;
4    --error-container: #93000a;
5    --on-background: #f0dee0;
6    --on-error: #690005;
7    --on-error-container: #ffdad6;
8    --on-primary: #561d2a;
9    --on-primary-container: #ffd9de;
10   --on-secondary: #43292d;
11   --on-secondary-container: #ffd9de;
12   --on-surface: #f0dee0;
13   --on-tertiary: #452b08;
14   --on-tertiary-container: #ffddba;
15   --primary: #ffb2bd;
16   --primary-container: #72333f;
17   --secondary: #e5bdc1;
18   --secondary-container: #5c3f43;
19   --surface: #191112;
20   --surface-container: #261d1e;
21   --tertiary: #eabf90;
22   --tertiary-container: #5f411c;
23 }
24
25 * {
26   color: var(--on-surface);
27   border: none;
28   outline: none;
29 }
30
31 text selection {
32   background: var(--tertiary);
33   color: var(--on-tertiary);
34 }

```

O servidor é responsável por iniciar a aplicação GTK e ouve por conexões em uma *socket unix*. A sua responsabilidade é executar o laço de eventos do GTK e gerenciar a criação dos *widgets*.

O cliente permite enviar mensagens ao servidor pela linha de comando. Isso é particularmente útil se o usuário quiser, por exemplo, recarregar um estilo customizado Sass sem ter que reiniciar o programa ou esconder temporariamente a barra de tarefas.

3.2.2 Serviços

Serviço é uma abstração formada por dois componentes: a implementação da funcionalidade central do serviço e o seu gerenciador.

A implementação do serviço é feita utilizando um trait chamado *Service*, ou interface, que possui as seguintes responsabilidades:

- Tratar mensagens de *Input* recebida dos clientes
- Enviar mensagens de *Output* para os clientes
- Gerar eventos internos (opcional)
- Tratar eventos internos (opcional)

A mensagem de *Input* atua como ações que os clientes mandam para o serviço e as mensagens de *Output* indicam mudanças de estado do serviço.

O gerenciador é uma estrutura de dados genérica que funciona para qualquer serviço que implementa o trait *Service*:

- *stop()*: Iniciar o serviço
- *start()*: Parar o serviço
- *message(...)*: Enviar mensagem para o serviço
- *listen(...)*: Permitir que clientes se inscrevam para receber mensagens de *Output*

Para facilitar o uso dos serviços e gerenciadores de serviço, foi implementada uma macro de tipo *derive* que cuida da criação de um gerenciador de serviço *singleton* e permite que funções do gerenciador sejam chamadas diretamente no objeto de serviço. Assim, o usuário só precisa se preocupar em implementar o *trait* de serviço e a macro cuida do resto. O exemplo 3.8 demonstra o seu uso em comparação à criação manual do gerenciador.

Nesse sentido, se quisermos criar um serviço que controla os dispositivos bluetooth, poderíamos ter algo como:

- *Input*: ligar/desligar bluetooth, conectar/desconectar dispositivo
- *Output*: estado de dispositivo bluetooth
- Tratar mensagens: recebe *Input* e altera o estado de algum dispositivo bluetooth
- Gerar eventos internos: ouve por mudanças no estado dos dispositivos bluetooth
- Tratar eventos internos: se estado do dispositivo mudou, envia mensagem de *Output*

Com isso, é possível usar essa abstração de serviço para descrever todas as funcionalidades de *desktop* do programa, desde controle de áudio e brilho até *daemons* de notificação D-Bus. O próximo capítulo aprofunda cada uma dessas funcionalidades.

Note também que o design dessa abstração de serviço é similar ao MVU usado pelo Relm4, porém sem o *View*. Isso permite que o *Model-Update* (o serviço) e o *View* (os *widgets* GTK) rodem em *threads* diferentes. Assim, o programa faz uso de múltiplas *threads* mesmo que o GTK seja *single-threaded*. Outra vantagem é que múltiplos *widgets* podem fazer uso de um mesmo serviço, isto é, uma comunicação de tipo 1:N (*publish-subscribe*).

Programa 3.8 Exemplo de uso da macro *derive StaticServiceManager* para implementar um gerenciador de serviço *singleton*.

```

1  // Esta macro derive adiciona métodos estáticos ao ServiceA
2  // que são delegados a ao gerenciador de serviços singleton
3  #[derive(azalea_derive::StaticServiceManager)]
4  pub struct ServiceA {
5      pub data: String
6  }
7
8  impl azalea_service::Service for ServiceA {
9      type Init = (); // dados de inicialização do servidor
10
11      type Input = ();
12      type Event = ();
13      type Output = ();
14
15      // ...
16  }
17
18  fn main() {
19      // A macro derive permite chamar os métodos estaticamente
20      // na struct do serviço
21      ServiceA::start();
22      ServiceA::stop();
23
24      // Sem a macro, seria necessário criar uma instância do gerenciador
25      // de serviço e emprestar essa referência a TODOS os widgets que
26      // fazem uso desse serviço...
27      // Ou criar ter que criar o singleton manualmente...
28      let serviceA_manager = ServiceManager::<ServiceA>::new(...);
29      serviceA_manager.start();
30      serviceA_manager.stop();
31
32      // OBS.: Quando um cliente deseja ouvir as mensagens de Output,
33      // ele recebe um "ListenerHandle". O conceito de Ownership é
34      // utilizado aqui para que no momento em que esse handle sai de
35      // escopo, a conexão é automaticamente fechada.
36      let listen_handle = ServiceA::listen(...);
37  }

```

Capítulo 4

Funcionalidades de *desktop*

Este capítulo trata das funcionalidades de *desktop* implementadas, ou seja, os *widgets* e serviços. Cada *widget* possui uma estrutura de configuração que permite configurar o *widget* a partir de um arquivo em formato JSON ou RON.¹ Para fazer a serialização e desserialização desse arquivo de configuração foi utilizada a biblioteca *serde* (<https://serde.rs/>).

A configuração de uma janela, como o papel de parede (4.1) ou barra de tarefas (4.2), possui a seguinte estrutura:

- `config`: configuração específica do *widget*
- `layer_shell`: permite configurar a camada, âncora, *namespace* e zona exclusiva
- `lazy`: Indica que o programa não deve criar esta janela durante a inicialização (isso só é útil se o usuário precisar de uma janela criada dinamicamente com o comando “`azalea window create [NOME]`”)
- `monitor`: Indica em qual monitor a janela deve ser criada

¹ RON (<https://docs.rs/ron/latest/ron/>) é uma notação cuja sintaxe se parece com código Rust

4.1 Papel de parede

O papel de parede é simplesmente uma janela que mostra uma imagem na camada de *background* e é ancorada aos quatro cantos da tela. Sua configuração também é simples, pois precisa receber apenas o caminho da imagem. Se não for especificado o caminho de alguma imagem, o programa usa o papel de parede padrão (4.1).



Figura 4.1: Papel de parede padrão^a

^a ilustração feita pelo autor

Programa 4.1 Exemplo de configuração do papel de parede em formato RON.

```
1  "wallpaper": (  
2      config: Wallpaper(  
3          image: None,  
4      ),  
5      layer_shell: (  
6          namespace: "wallpaper",  
7          layer: Background,  
8          anchors: [  
9              Left,  
10             Right,  
11             Bottom,  
12             Top,  
13         ],  
14         exclusive_zone: Ignore,  
15     ),  
16     lazy: false,  
17     monitor: All,  
18 )
```

4.2 Barra de tarefas

A barra de tarefas implementada permite customizar a ordem dos widgets na barra. A barra é dividida em 3 áreas: início, centro e fim; e cada área pode possuir múltiplos *widgets*. O exemplo 4.2 mostra a configuração padrão utilizada para a barra de tarefas do canto inferior da tela.

Note que a partir do arquivo de configuração é extremamente fácil mover essa barra de tarefas para o topo, basta mudar a âncora da propriedade *layer_shell* e recarregar o programa.

Programa 4.2 Exemplo de configuração da barra de tarefas do canto inferior da tela em formato RON.

```

1  "bottom-taskbar": (
2      config: Taskbar(
3          spacing: 8,
4          start: [
5              StartMenu(),
6              Audio(),
7              Brightness(),
8          ],
9          center: [
10             Media(),
11         ],
12         end: [
13             Bluetooth(),
14             Network(),
15             Separator(separator: None),
16             Time(
17                 format: "%d/%m/%y",
18             ),
19             Separator(separator: None),
20             Time(
21                 format: "%H:%M",
22             ),
23         ],
24     ),
25     layer_shell: (
26         namespace: "taskbar",
27         layer: Top,
28         anchors: [
29             Left,
30             Right,
31             Bottom, # canto inferior da tela
32         ],
33         exclusive_zone: Auto,
34     ),
35     lazy: false,
36     monitor: All,
37 ),

```

Várias das implementações nas subseções seguintes utilizam a abstração de serviço

explicada em 3.2.2. Em suma, os serviços podem ser descritos pelos tipos de mensagem que ele recebe dos clientes (*Input*) e as mensagens que envia por *broadcast* a todos os clientes (*Output*). Em casos especiais há também eventos internos, mas os clientes (*widgets*) não precisam saber disso para usar o serviço.

4.2.1 Controle de volume

O controle de volume foi implementado utilizando ALSA e possui mensagens bem simples:

- `Input::SystemVolume(f64)`: Requisição para mudar o volume
- `Output::SystemVolume(f64)`: Novo valor do volume *broadcasted* para os clientes

O *widget* que usa esse serviço consiste em um *slider* que controla o volume (4.2).



Figura 4.2: Slider de volume

4.2.2 Controle de brilho

O controle de brilho é semelhante ao de áudio, porém utilizando a biblioteca *brightness* (<https://crates.io/crates/brightness>), que permite controlar o brilho por ACPI dos dispositivos em “/sys/class/backlight”. Isso funciona em telas de notebook, para monitores externos que utilizam DDC/CI seria necessário ter o driver *ddcci-backlight* (<https://gitlab.com/ddcci-driver-linux/ddcci-driver-linux>).

- `Input::SystemBrightness(f64)`: Requisição para mudar o brilho
- `Output::SystemBrightness(f64)`: Novo valor do volume *broadcasted* para os clientes

Analogamente ao *widget* anterior, o controle de brilho também foi implementado como um *slider* (4.3).



Figura 4.3: Slider de brilho

4.2.3 Data e hora

Este *widget* recebe o formato da data por meio da configuração (4.3) e mostra na tela a data e/ou hora. Ao clicar nessa data, ele cria um *popup* com o calendário padrão da biblioteca GTK (4.4).

Programa 4.3 Exemplo de configuração do *widget* de data e hora RON.

```

1  Time(
2    format: "%A, %B %d, %Y",
3  )

```

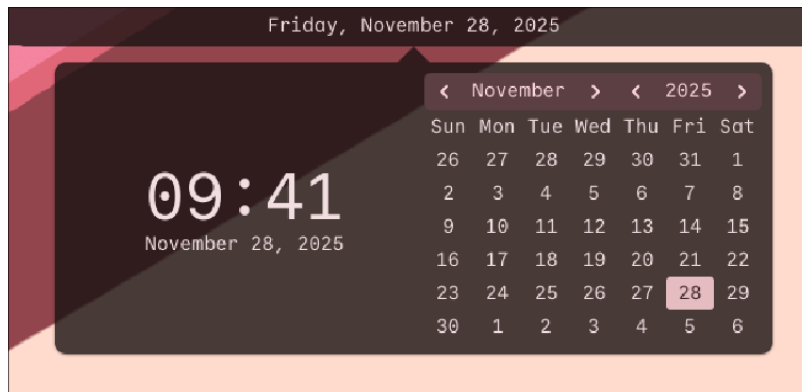


Figura 4.4: Widget de data e hora

4.2.4 Menu de início

O menu de início mostra a logo do projeto; botões para suspender, reiniciar, desligar e hibernar o computador e mostra informações do sistema.

Para os botões, foi implementado um serviço básico com um *proxy* para chamar os seguintes métodos da interface `org.freedesktop.login1`:

- `power_off`
- `reboot`
- `suspend`
- `hibernate`

Para pegar as informações do sistema foi utilizado a biblioteca *ffetch* (<https://crates.io/crates/ffetch>).

4.2.5 Controle de dispositivos *bluetooth*

Para lidar com *bluetooth* foi utilizada a *crate* BlueR (<https://crates.io/crates/bluer>), que são as *bindings* oficiais do BlueZ. Assim, para controlar os dispositivos *bluetooth*, foi criado um serviço que recebe mensagens de *Input* de tipo:

- `Power(bool)`: Pede para ligar ou desligar o *bluetooth*
- `Devices(canál de resposta)`: Pede lista de dispositivos e envia para o canal de resposta do cliente
- `Connect(DeviceAdress, bool)`: Pede para conectar ou desconectar o dispositivo

E mensagens *Output* de tipo:



Figura 4.5: Widget do menu de início.

- `Powered(bool)`: Avisa se o *bluetooth* foi ligado ou desligado
- `Connected(DeviceAddress, bool)`: Avisa se o dispositivo foi conectado ou não

Com isso, foi possível fazer o *widget* da figura 4.6 com a lista de dispositivos *bluetooth* e botões para conectar ou desconectar o dispositivo.

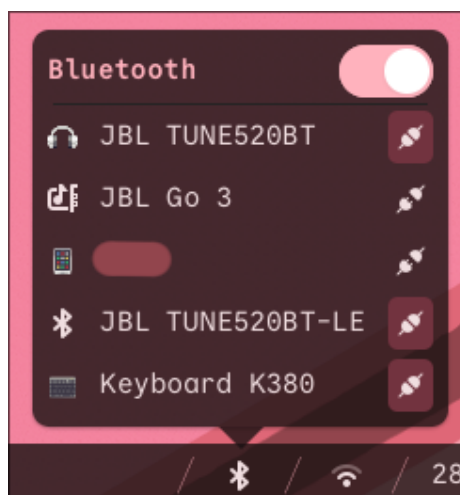


Figura 4.6: Widget de controle de dispositivos bluetooth

4.2.6 Controle de conexões de rede

Para controlar as conexões de rede, foi utilizado D-Bus. Mais especificamente, os objetos de “org.freedesktop.NetworkManager”, que, como o nome sugere, depende do programa *NetworkManager*.

O serviço recebe *Inputs* de tipo:

- `Enable(bool)`: liga ou desliga o `NetworkManager`
- `ListConnections`: lista as conexões
- `DeactivateConnection { ... }`: desativa alguma conexão
- `ActivateConnection { ... }`: ativa alguma conexão

E enviar *Outputs* avisando os clientes sobre mudanças no `NetworkManager`:

- `Connections(Vec<...>)`: Lista de conexões como resposta ao `Input::ListConnections`
- `NetworkingEnabledChanged(bool)`: Diz se o `NetworkManager` foi ativado ou desativado
- `StateChanged(NMState)`: Avisa sobre mudança no estado na rede (e.g. está conectado, desconectado, prestes a conectar, etc)
- `ConnectivityChanged(NMConnectivityState)`: Avisa sobre mudança de estado da conexão (e.g. se está conectado, se tem acesso à internet, etc.)

Assim, foi possível criar o *widget* da figura 4.7. O botão na barra de tarefas mostra um ícone diferente na barra de tarefas dependendo do estado da conexão. Quando esse botão é clicado, um *popup* é aberto com a lista de conexões conhecidas.

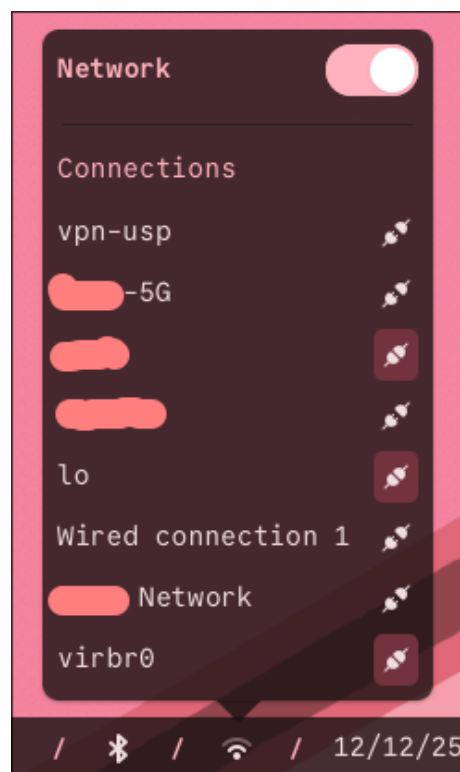


Figura 4.7: *Widget de controle de conexões rede*

4.2.7 Notificações

Notificações no *desktop* utilizam o D-Bus. Então, foi necessário criar um *daemon* de notificações D-Bus que se conecta no barramento com o nome de “org.freedesktop.Notifications”. Essa conexão deve oferecer um objeto “/org/freedesktop/Notifications” que implementa a interface “org.freedesktop.Notifications”. Um exemplo básico de *daemon* de notificações implementado com zbus foi mencionado no exemplo 3.6.

Quando alguma aplicação quer enviar uma notificação, ele encontra a conexão de nome “org.freedesktop.Notifications” no barramento e chama o método de `notify()` do objeto “/org/freedesktop/Notifications”. Quando o *daemon* recebe essa mensagem, ele faz *broadcast* dessa notificações para os *widgets* clientes utilizando a mensagem de *Output*.

Quando o *widget* recebe essa mensagem, ele cria um *widget* que representa essa notificação e adiciona à sua lista vertical de notificações (4.8).



Figura 4.8: Widget de notificações

4.2.8 Controle de mídia

Aplicações que tocam vídeo ou algum tipo de mídia utilizam o padrão MPRIS do D-Bus para enviar informações de seus *players* de mídia. Nesse sentido, foi necessário

implementar um *proxy* para a interface D-Bus “org.mpris.MediaPlayer2.Player” e ficar ouvindo por novas conexões de *players* MPRIS.

O serviço envia *Outputs* com o nome do *player* e um evento de tipo:

- Volume(f64)
- Position(i64)
- Metadata(Metadata)
- PlaybackStatus(PlaybackStatus)
- PlaybackRate(PlaybackRate)

Isso permite que o cliente saiba o estado de vários *players* ao mesmo tempo. Analogamente, os *Inputs* também permitem dizer qual *player* deve ser alterado:

- PlayPause(nome do *player*)
- Previous(nome do *player*)
- Next(nome do *player*)

Isso foi suficiente para implementar o *widget* da figura 4.9, com seletor de *player* botões para o controle de mídia.

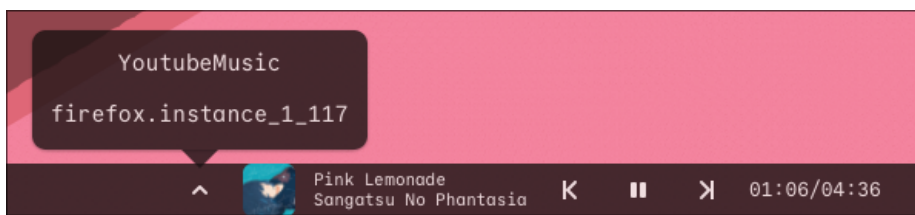


Figura 4.9: Widget MPRIS

4.2.9 Shortcuts

O *widget* de *shortcut* consiste em um botão que representa alguma *desktop entry*, isto é, aqueles arquivos *.desktop* que descrevem o nome, ícone, executável, descrição, entre outras informações de uma aplicação. Não foi necessário criar um serviço, bastou utilizar a estrutura *AppInfo* da biblioteca *Gio*, incluída no *crate* do *GTK*. A sua configuração também é simples e só precisa do nome da *desktop entry* (??).

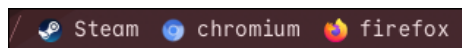


Figura 4.10: Widgets de shortcut para Steam, Chromium e Firefox

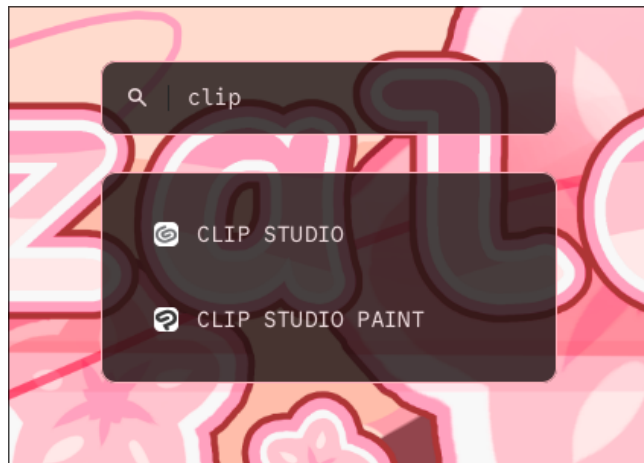
4.2.10 Barra de busca de aplicações

De forma semelhante aos *shortcuts*, foi utilizada a estrutura *AppInfo* do *Gio* para listar todas as aplicações.

Programa 4.4 Exemplo de configuração de *shortcut* em formato RON.

```
1 Shortcut(  
2   desktop_entry: "firefox.desktop",  
3 )
```

Este *widget* tem uma implementação experimental em que o *widget* na barra de tarefas (4.11b) cria, ao ser clicado, a janela com a barra de pesquisa (4.11a). Também seria possível ter implementado isso como uma janela à parte, como o papel de parede e da barra de tarefas.



(a) Barra de busca de aplicações



(b) Botão na barra de tarefas

Figura 4.11: Widget de busca de aplicações

4.2.11 System tray

Esta funcionalidade acabou não sendo implementada. Existem vários protocolos diferentes e ainda não há uma especificação de fato padrão:

- *Protocolo de System Tray (freedesktop)*: específico do X11, utiliza XEMBED e é considerado *legacy* (mas ainda é utilizado)
- *KStatusNotifierItem (KDE)*: utiliza D-Bus, não depende do servidor de display
- *StatusNotifierItem (freedesktop)*: baseado na do KDE, porém ainda está em estado de rascunho
- *Ayatana App Indicator*: baseado no StatusNotifierItem, mas como é recente, não possui adoção ampla no momento

Em teoria, a especificação do *freedesktop* deveria ser o padrão. Porém, na prática, a maioria dos clientes implementa o protocolo do KDE. Nesse sentido, o *System Tray* acabou não sendo implementado.

4.3 Overview

A figura 4.12 mostra um *overview* de todos os *widgets*.



Figura 4.12: Colagem^a de fotos com todos os *widgets*

^a Tecnicamente, não é possível mostrar todos os *widgets* ao mesmo tempo, pois o GTK fecha os *popups* quando eles perdem foco. Nesse sentido, a imagem é uma colagem de fotos dos *widgets*.

Capítulo 5

Nix

Nix (<https://nixos.org/>) é uma linguagem funcional que permite criar ambientes de desenvolvimento, empacotar aplicações, configurar o sistema, etc, de forma declarativa.

5.0.1 Ambiente de desenvolvimento

O ambiente de desenvolvimento, ou *dev shell*, com todas as ferramentas, bibliotecas e LSP, foi configurado utilizando Nix. O exemplo 5.1 mostra como esse shell é usado na prática e 5.2 mostra uma implementação simples de um *dev shell*.

Programa 5.1 Exemplo de como usar um *dev shell* nix.

```
1  # Inicialmente, rustc não está disponível no PATH
2  $ rustc --version
3  bash: line 1: rustc: command not found
4
5  # Ao entrar no dev shell, rustc se torna disponível para uso
6  $ nix develop
7  $ rustc --version
8  rustc 1.91.1 (ed61e7d7e 2025-11-07) (built from a source tarball)
9
10 # Se sair do dev shell, rustc volta a não estar disponível
11 $ exit
12 $ rustc --version
13 bash: line 1: rustc: command not found
```

5.0.2 Empacotamento

Nix também foi usado para empacotar o programa. Isso permite que ele seja construído pelo comando “nix build” sem precisar se preocupar em instalar as bibliotecas no sistema. O exemplo 5.3 mostra como as dependências, *patches*, código-fonte, etc, são declarados.

Programa 5.2 Exemplo de *dev shell* nix.

```
1  # Lista de argumentos
2  {
3    mkShell,
4    cargo,
5    rustc,
6    rustfmt,
7    rust-analyzer,
8  }:
9
10 # Cria dev shell
11 mkShell {
12   name = "example-devshell";
13
14   # Variáveis de ambiente
15   RUST_SRC_PATH = "${pkgs.rust.packages.stable.rustPlatform.rustLibSrc}";
16
17   # Lista de pacotes que devem ser incluídos no shell
18   # compiladores, bibliotecas, formatadores, LSPs, etc
19   buildInputs = [
20     cargo
21     rustc
22     rustfmt
23     rust-analyzer
24   ];
25 }
```

5.0.3 Testes em máquinas virtuais NixOS

O nix também pode ser usado para criar testes automatizados que criam máquinas virtuais NixOS,¹ tiram uma *screenshot* e salvam os resultados na pasta `result` (5.4).

Idealmente, é desejável testar a aplicação em compositores Wayland que usam bibliotecas diferentes. Nesse sentido, foram escolhidas os seguintes compositores:

- wlroots: Sway (5.1), Hyprland (5.2), Wayfire (5.3)
- smithay: Niri (5.5)²
- Mir: Miracle (5.4)

¹ Mais informações sobre testes em máquinas virtuais NixOS, veja a *wiki*: https://wiki.nixos.org/wiki/NixOS_VM_tests

² No caso do Niri em particular, foi necessário rodar o teste manualmente com *hardware rendering*, pois Niri não suporta *software rendering*: <https://github.com/YaLTeR/niri/issues/218>



Figura 5.1: Azalea rodando no Sway em máquina virtual NixOS.

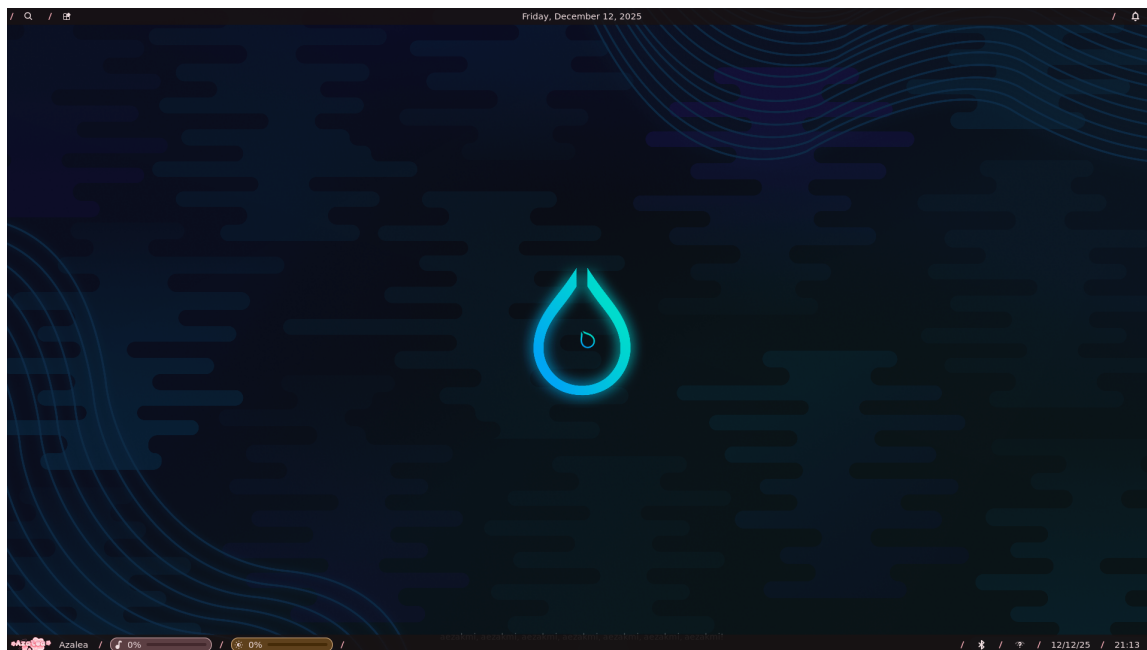


Figura 5.2: Azalea rodando no Hyprland em máquina virtual NixOS.

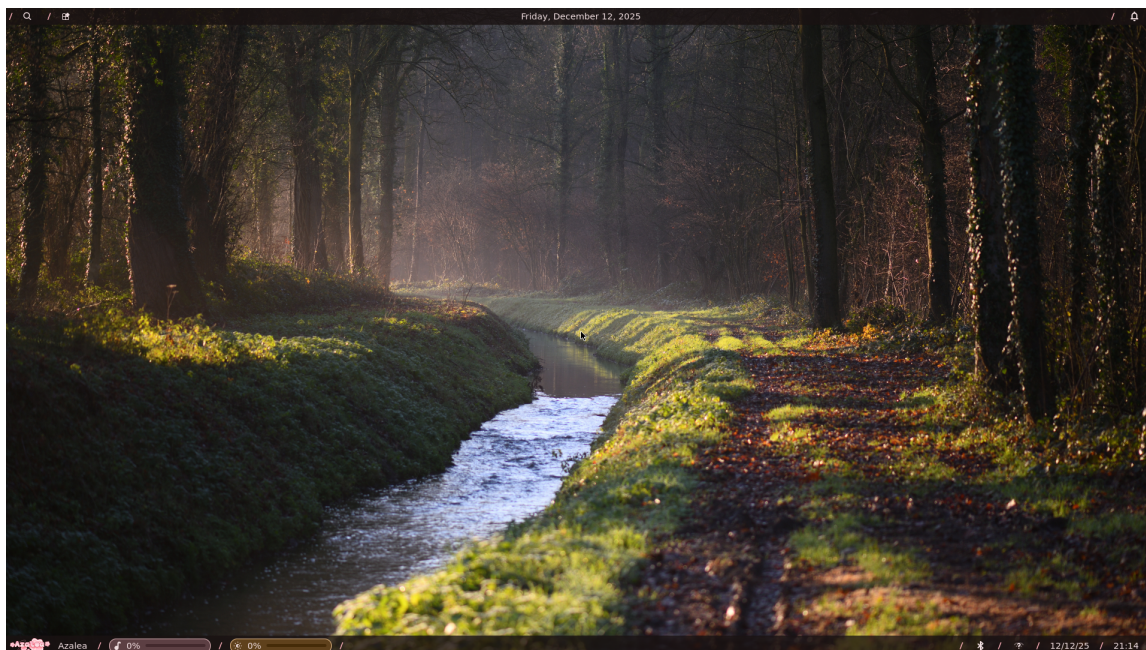


Figura 5.3: Azalea rodando no Wayfire em máquina virtual NixOS.



Figura 5.4: Azalea rodando no Miracle em máquina virtual NixOS.

Programa 5.3 Exemplo simples de como empacotar uma aplicação fantasia em nix.

```
1  # Lista de argumentos
2  {
3      stdenv,
4      fetchurl,
5      fetchFromGitHub,
6
7      gtk4
8      gtk4-layer-shell
9  }:
10
11 # Cria a "derivação" (pacote)
12 stdenv.mkDerivation (finalAttrs: {
13     pname = "example";
14     version = "2.3.4";
15
16     # Código fonte
17     src = fetchFromGitHub {
18         owner = "enzohideo";
19         repo = "some-repository";
20         rev = "90bb10e1564c8331b6709957943e95e8cd586769";
21         hash = "sha256-WpqZbcKSzCTc9ASB6H6S9urLuNE72caBm0x6nc4IGKs=";
22     };
23
24     # Aplicar patches
25     patches = [
26         ./00-fix-something.patch
27     ];
28
29     # Bibliotecas
30     buildInputs = [
31         gtk4
32         gtk4-layer-shell
33     ];
34 })
```

Programa 5.4 Exemplo de teste em máquina virtual NixOS

```
1  testers.runNixOSTest {
2    name = "azalea-integration-test";
3
4    # Inicia duas máquinas virtuais com configurações customizadas
5    nodes.hyprrland = import ./nodes/hyprrland.nix;
6    nodes.sway = import ./nodes/sway.nix;
7
8    # Configuração padrão utilizadas em todas as máquinas virtuais
9    defaults = {
10      virtualisation.qemu.options = [ "-vga virtio" ];
11    };
12
13    # O script de teste é em Python
14    testScript = ''
15      def test(machine):
16        machine.start()
17
18        machine.wait_for_unit("multi-user.target")
19        machine.wait_until_succeeds("pgrep azalea")
20        machine.sleep(10)
21
22        with subtest(f"{machine.name}: default"):
23          machine.screenshot(f"{machine.name}-default")
24
25        machine.shutdown()
26
27      for machine in machines:
28        with subtest(machine.name):
29          test(machine)
30    '';
31  }
```

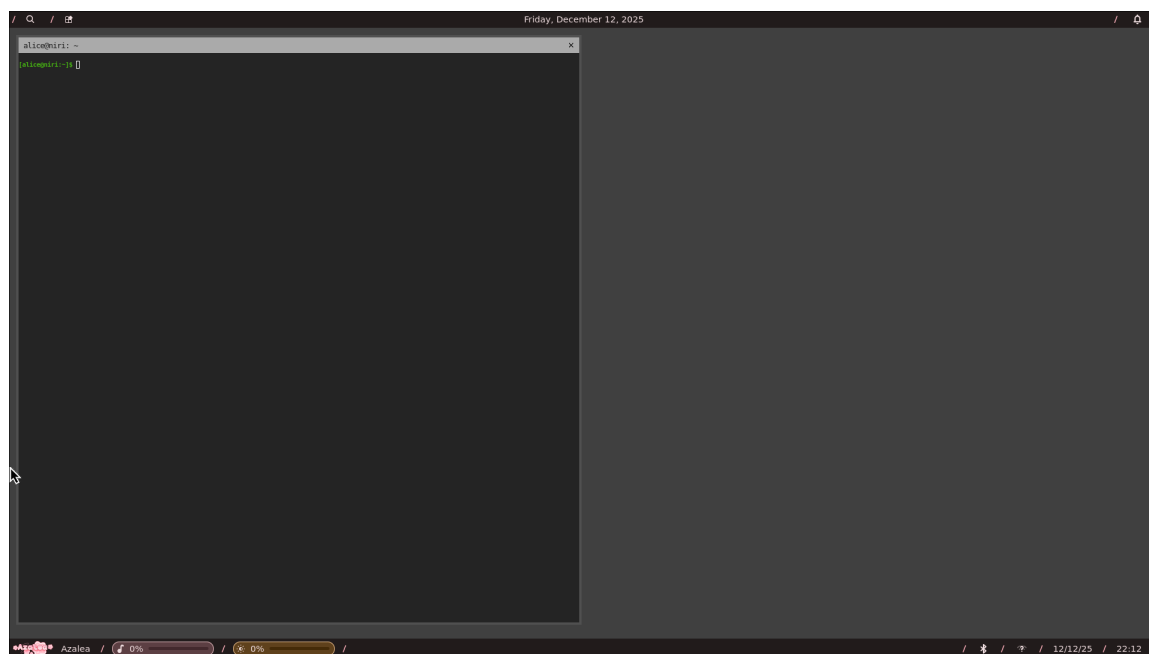


Figura 5.5: Azalea rodando no Niri em máquina virtual NixOS.

Capítulo 6

Conclusão

O resultado deste trabalho pode ser encontrado em <https://github.com/enzohideo/azalea-desktop-shell>. Foram desenvolvidas as seguintes *crates*:

- *azalea*: aplicação principal e re-exportação das bibliotecas internas
- *azalea-core*: cliente-servidor, CLI, estilo Sass, *parsing* dos arquivos de configuração, etc
- *azalea-log*: funções básicas de *logging*
- *azalea-service*: trait do serviço e gerenciador de serviço
- *azalea-derive*: macros *derive*
- *azalea-shell*: implementação dos serviços e *widgets*

A crate *azalea* é responsável por gerar o binário do programa, mas também pode ser importado e usado como biblioteca para implementar outros *shells* gráficos. Sua documentação foi colocada no GitHub Pages do repositório (<https://enzohideo.github.io/azalea-desktop-shell/docs/azalea/>)

A maioria das funcionalidades esperadas foram implementadas, com exceção do *System Tray*, como comentado na seção 4.2.11. Existem também alguns outros protocolos interessantes que não foram explorados neste projeto, mas que poderiam ser em trabalhos futuros:

- *ext-session-lock-v1*: Protocolo Wayland que permite criar *lockscreens*
- *ext-workspace-v1*: Protocolo Wayland para gerenciar *workspaces* (relativamente recente)
- *greetd-ipc*: Protocolo específico do *greetd* que permite a criação de gerenciadores de login

Por fim, em relação ao objetivo central do projeto de funcionar em múltiplos compositores, pode-se concluir que foi bem-sucedido.

Referências

- [FREEDESKTOP 2025a] FREEDESKTOP. *D-Bus*. <https://www.freedesktop.org/wiki/Software/dbus/>. Acessado em dezembro. 2025 (citado na pg. 5).
- [FREEDESKTOP 2025b] FREEDESKTOP. *Wayland*. <https://wayland.freedesktop.org/>. Acessado em dezembro. 2025 (citado na pg. 3).
- [FREEDESKTOP 2025c] FREEDESKTOP. *Wayland Protocols*. <https://gitlab.freedesktop.org/wayland/wayland-protocols>. Acessado em dezembro. 2025 (citado na pg. 4).
- [RELM4 CONTRIBUTORS 2025] RELM4 CONTRIBUTORS. *GUI development with Relm4*. <https://relm4.org/book/stable/introduction.html>. Acessado em dezembro. 2025 (citado na pg. 11).
- [STEVE KLABNIK, CAROL NICHOLS, CHRIS KRYCHO, RUST COMMUNITY CONTRIBUTORS 2025] STEVE KLABNIK, CAROL NICHOLS, CHRIS KRYCHO, RUST COMMUNITY CONTRIBUTORS. *The Rust Programming Language, Third Edition*. <https://doc.rust-lang.org/stable/book/>. Acessado em dezembro. 2025 (citado na pg. 7).
- [WAYLAND EXPLORER 2025] WAYLAND EXPLORER. *wlr layer shell*. <https://wayland.app/protocols/wlr-layer-shell-unstable-v1>. Acessado em dezembro. 2025 (citado na pg. 4).
- [WIKIPEDIA CONTRIBUTORS 2025] WIKIPEDIA CONTRIBUTORS. *Shell (computing)* — *Wikipedia, The Free Encyclopedia*. [https://en.wikipedia.org/w/index.php?title=Shell_\(computing\)&oldid=1325918852](https://en.wikipedia.org/w/index.php?title=Shell_(computing)&oldid=1325918852). Acessado em dezembro. 2025 (citado na pg. 3).